Parsers Funcionales

Juan Manuel Rabasedas

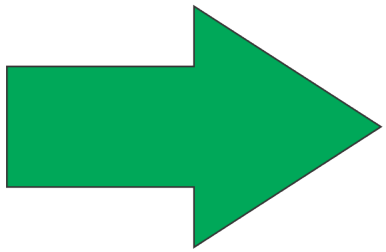
¯_(ツ)_/¯

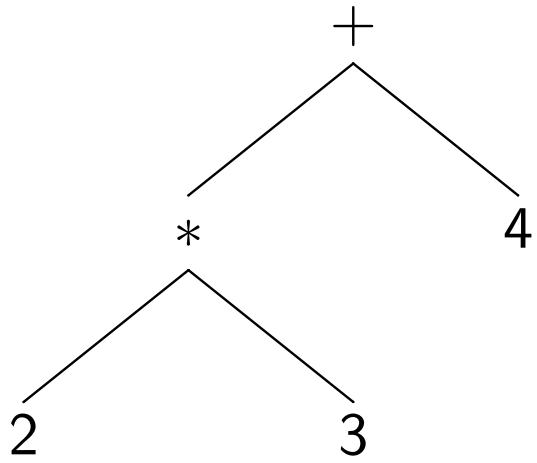
¿Qué es un parser?

Un parser es un programa que toma un String como entrada y produce algún tipo de árbol como salida. La idea es que el árbol explicite la estructura sintáctica en la cadena de entrada.

$2 * 3 + 4$

Parser





El tipo Parser

En los lenguajes funcionales un parser puede verse como una función

$$\textbf{type}\ \text{Parser} = \text{String} \rightarrow \text{Tree}$$

que toma un String y retorna algún tipo de árbol

Por lo general un Parser no consume toda su entrada, así que también podemos retornar la parte de la entrada que no se uso

$$\textbf{type}\ \text{Parser} = \text{String} \rightarrow (\text{Tree}, \text{String})$$

El tipo Parser

Por otro lado una cadena puede ser parseada de muchas maneras, incluso de ninguna. Será conveniente generalizarlo a una lista de posibles resultados.

$$\textbf{type}\ \text{Parser} = \text{String} \rightarrow [(\text{Tree}, \text{String})]$$

Finalmente un parser podría producir algo que no es un árbol, así que lo generalizaremos para un valor de cualquier tipo

$$\textbf{type}\ \text{Parser}\ a = \text{String} \rightarrow [(a, \text{String})]$$

Por simplicidad solo vamos a considerar a parsers que fallan y retornan una lista vacía o que tienen éxito y retornan una lista con un sólo elemento.

Parsers básicos

El parser *item* falla cuando la entrada es vacía, si no lo es consume el primer caracter.

$$
\begin{aligned}
&\mathit{item} :: \text{Parser Char} \\
&\mathit{item} = \lambda \mathit{inp} \rightarrow \textbf{case } \mathit{inp} \textbf{ of} \\
&\qquad\qquad\qquad\qquad [\,] \quad\;\; \rightarrow [\,] \\
&\qquad\qquad\qquad\qquad (x : xs) \rightarrow [(x, xs)]
\end{aligned}
$$

Parsers básicos

El parser *failure* siempre falla.

$$failure :: \text{Parser } a$$
$$failure = \lambda inp \rightarrow [\,]$$

El parser *return v* siempre tiene éxito y retorna el valor v sin consumir la entrada.

$$return :: a \rightarrow \text{Parser } a$$
$$return\ v = \lambda inp \rightarrow [(v, inp)]$$

Parsers básicos

El parser $p <|> q$ se comporta como el parser p si este tiene éxito y como el parser q si falla

$$
\begin{aligned}
&<|> :: \text{Parser } a \to \text{Parser } a \to \text{Parser } a \\
&p <|> q = \lambda inp \to \textbf{case } p \ inp \ \textbf{of} \\
&\qquad\qquad [\,] \qquad\quad \to parse \ q \ inp \\
&\qquad\qquad [(v, out)] \to [(v, out)]
\end{aligned}
$$

La función *parse* aplica un parser a una cadena

$$
\begin{aligned}
&parse :: \text{Parser } a \to \text{String} \to [(a, \text{String})] \\
&parse \ p \ inp = p \ inp
\end{aligned}
$$

Ejemplos

El comportamiento de las cinco primitivas puede ser ilustrado con algunos ejemplos simples:

```
% ghci Parsing
> parse item ""
[]


> parse item "abc"
[('a',"bc")]
```

Ejemplos

```
> parse failure "abc"
[]
```

```
> parse (return 1) "abc"
[(1,"abc")]
```

```
> parse (item <|> return 'd') "abc"
[('a',"bc")]
```

```
> parse (failure <|> return 'd') "abc"
[('d',"abc")]
```

Nota

- La librería Parsing está disponible en la página de la materia
- Por simplicidad algunos parser fueron modificados para la presentación.
- El tipo Parser es una mónada, una estructura matemática que ha demostrado ser de utilidad para modelar diferentes tipos de computaciones.

Secuenciación

Una secuencia de parsers puede ser combinada en una único parser usando la palabra reservada **do**:

$$
\begin{aligned}
&p :: \text{Parser } (\text{Char}, \text{Char}) \\
&p = \textbf{do } x \leftarrow \mathit{item} \\
&\qquad\quad \mathit{item} \\
&\qquad\quad y \leftarrow \mathit{item} \\
&\qquad\quad \mathit{return}\ (x, y)
\end{aligned}
$$

Secuenciación

- Cada parser debe comenzar en la misma columna.
- Los valores devueltos por los parser intermedios son descartados, pero si se requiere pueden ser nombrados usando el operador $\leftarrow$
- El valor retornado por el último parser es el valor de retorno de toda la secuencia.
- Si algún parser de la secuencia falla, la secuencia completa falla:

$>$ *parse p* `"abcdef"`
`[(('a','c'),"def")]`

$>$ *parse p* `"ab"`
`[]`

La notación **do** *no es especifica del tipo* Parser, *puede ser usada con cualquier tipo monádico.*

Derivando Primitivas

Parsear un caracter que satisface un predicado:

```
sat :: (Char → Bool) → Parser Char
sat p = do x ← item
           if p x then
              return x
           else
              failure
```

Parsear un dígito y un caracter específico

```
digit :: Parser Char
digit = sat isDigit

char :: Char → Parser Char
char x = sat (x ≡)
```

Derivando Primitivas

Aplicar un parser cero o más veces

$$
\begin{aligned}
&\mathit{many} :: \text{Parser } a \to \text{Parser } [a] \\
&\mathit{many}\ p = \mathit{many1}\ p < | > \mathit{return}\ [\,]
\end{aligned}
$$

Aplicar un parser una o más veces

$$
\begin{aligned}
&\mathit{many1} :: \text{Parser } a \to \text{Parser } [a] \\
&\mathit{many1}\ p = \textbf{do}\ v \leftarrow p \\
&\qquad\qquad\qquad\quad vs \leftarrow \mathit{many}\ p \\
&\qquad\qquad\qquad\quad \mathit{return}\ (x : vs)
\end{aligned}
$$

Parseando una cadena de caracteres específica:

$$
\begin{aligned}
&\mathit{string} :: \text{String} \to \text{Parser String} \\
&\mathit{string}\ [\,] \qquad\quad = \mathit{return}\ [\,] \\
&\mathit{string}\ (x : xs) = \textbf{do}\ \mathit{char}\ x \\
&\qquad\qquad\qquad\qquad\quad \mathit{string}\ xs \\
&\qquad\qquad\qquad\qquad\quad \mathit{return}\ (x : xs)
\end{aligned}
$$

Ejemplo

Podemos definir un parser que reconozca una lista de números en un String:

```
p :: Parser String
p = do char '['
       d ← digit
       ds ← many (do char ','
                     digit)
       char ']'
          return (d : ds)
```

```
> parse p "[1,2,3,4]"
[("1234", "")]
> parse p "[1,2,3,4"
[]
```

Una más sofisticada biblioteca de parsing pude indicar y/o recuperarse de errores en la cadena de entrada.

Expresiones Aritméticas

Considere una forma simple de expresiones construidas a partir de dígitos usando las operaciones de suma $+$ y multiplicación $*$ junto con paréntesis. También asumiremos que:

- el $*$ y $+$ asocian hacia la derecha
- y que $*$ tiene más prioridad que la $+$

Formalmente la sintaxis de estas expresiones está definida por la siguiente gramática libre de contexto:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{term}\ \text{'+'}\ \mathit{expr} \mid \mathit{term} \\
\mathit{term} &\rightarrow \mathit{factor}\ \text{'*'}\ \mathit{term} \mid \mathit{factor} \\
\mathit{factor} &\rightarrow \mathit{digit} \mid \text{'('}\ \mathit{expr}\ \text{')'} \\
\mathit{digit} &\rightarrow \text{'0'} \mid \text{'1'} \mid \text{'2'} \mid \ldots \mid \text{'9'}
\end{aligned}
$$

Expresiones Aritméticas

Sin embargo, por razones de eficiencia, es importante factorizar las reglas para *expr* y *term*:

$$expr \rightarrow term\ (\text{'+'}\ expr \mid \varepsilon)$$
$$term \rightarrow factor\ (\text{'*'}\ term \mid \varepsilon)$$

Ahora es fácil traducir la gramática a un parser que evalúa expresiones, simplemente reescribiendo las reglas de la gramática usando las primitivas de parseo.

$$expr \rightarrow term\ (\text{'+'}\ expr \mid \varepsilon)$$

```
expr :: Parser Int
expr = do t ← term
          do char '+'
             e ← expr
             return (t + e)
          <|> return t
```

Expresiones Aritméticas

$term \rightarrow factor\ (\text{'*'}\ term \mid \varepsilon)$

```
term :: Parser Int
term = do f ← factor
          do char '*'
             t ← term
             return (f * t)
           <|> return f
```

$factor \rightarrow digit \mid \text{'('}\ expr\ \text{')'}$

```
factor :: Parser Int
factor = do d ← digit
            return (digitToInt d)
          <|> do char '('
                 e ← expr
                 char ')'
                 return e
```

Expresiones Aritméticas

Finalmente definimos:

$$eval :: \text{String} \to \text{Int}$$
$$eval\ xs = fst\ (head\ (parse\ expr\ xs))$$

Probamos algunos ejemplos

```
> eval "2*3+4"
10
> eval "2*(3+4)"
14
```

Ejercicios

1. ¿Por qué factorizar la gramática hace que el parser sea más eficiente?
2. Extender el parser de expresiones para usar la resta y la división, basándose en la siguiente gramática:

$$expr \rightarrow term\ (\text{'+'}\ expr \mid \text{'-'}\ expr \mid \varepsilon)$$
$$term \rightarrow factor\ (\text{'*'}\ term \mid \text{'/'}\ term \mid \varepsilon)$$

Ejercicios

Consideremos ahora construir un evaluador de expresiones aritméticas donde el operador de sustracción asocie a izquierda. La gramática sería la siguiente:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{expr}\ \text{'+'}\ \mathit{term} \mid \mathit{expr}\ \text{'-'}\ \mathit{term} \mid \mathit{term} \\
\mathit{term} &\rightarrow \mathit{term}\ \text{'*'}\ \mathit{factor} \mid \mathit{term}\ \text{'/'}\ \mathit{factor} \mid \mathit{factor} \\
\mathit{factor} &\rightarrow \mathit{digit} \mid \text{'('}\ \mathit{expr}\ \text{')'} \\
\mathit{digit} &\rightarrow \text{'0'} \mid \text{'1'} \mid \text{'2'} \mid \ldots \mid \text{'9'}
\end{aligned}
$$

Luego de ser factorizada, la gramática queda de la siguiente forma:

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{expr}\ (\text{'+'}\ \mathit{term} \mid \text{'-'}\ \mathit{term}) \mid \mathit{term} \\
\mathit{term} &\rightarrow \mathit{term}\ (\text{'*'}\ \mathit{factor} \mid \text{'/'}\ \mathit{factor}) \mid \mathit{factor}
\end{aligned}
$$

Ejercicios

El parser correspondiente a la producción

$$expr \rightarrow expr\ \text{'+'}\ term \mid term$$

sería:

$$\begin{aligned} &expr :: \text{Parser Int} \\ &expr = \textbf{do}\ t \leftarrow expr \\ &\qquad\qquad char\ \text{'+'} \\ &\qquad\qquad e \leftarrow term \\ &\qquad\qquad return\ (t + e) \\ &\quad < \mid > term \end{aligned}$$

¿Cuál es el problema con este parser?

Ejercicios

El parser correspondiente a la producción

$$expr \rightarrow \boxed{\text{expr}}\ \text{'+'}\ term \mid term$$

sería:

```
expr :: Parser Int
expr = do t ← expr
          char '+'
          e ← term
          return (t + e)
       <|> term
```

Esto se conoce como problema de la recursión izquierda.

Solución

Podemos transformar la gramática en una equivalente que no tenga recursión izquierda:
Dada una gramática:

$$A \rightarrow A\ \alpha \mid \beta$$

tal que α es no vacío y β no empieza con A
Podemos transformarla de la siguiente manera:

$$A \rightarrow \beta\ A'$$
$$A' \rightarrow \varepsilon \mid \alpha\ A'$$

Por ejemplo:

$$\textit{resto} \rightarrow \textit{resto}\ \texttt{'sigue'}\ \textit{resto} \mid \texttt{'valor'}$$

se transforma a

$$\textit{resto} \rightarrow \texttt{'valor'}\ \textit{resto}'$$
$$\textit{resto}' \rightarrow \varepsilon \mid \texttt{'sigue'}\ \textit{resto}\ \textit{resto}'$$

Ejercicio

Transformar la gramática del ejercicio para eliminar la recursión a izquierda

$$
\begin{aligned}
\mathit{expr} &\rightarrow \mathit{expr}\ (\text{'+'}\ \mathit{term} \mid \text{'-'}\ \mathit{term}) \mid \mathit{term} \\
\mathit{term} &\rightarrow \mathit{term}\ (\text{'*'}\ \mathit{factor} \mid \text{'/'}\ \mathit{factor}) \mid \mathit{factor} \\
\mathit{factor} &\rightarrow \mathit{digit} \mid \text{'('}\ \mathit{expr}\ \text{')'} \\
\mathit{digit} &\rightarrow \text{'0'} \mid \text{'1'} \mid \text{'2'} \mid \ldots \mid \text{'9'}
\end{aligned}
$$

Resumen

- Un parser nos permite reconocer la estructura sintáctica de una cadena de caracteres.
- Es fácil de implementar un parser complejo combinando o secuenciando parsers básico
- Debemos factoriazar o transformar las gramáticas para optimizar o eliminar recursión izquierda.

Bibliografía

- Programming in Haskell. G. Hutton. Cambridge University Press (2007). Capítulo 8.
- Moderm Compiler implementarion in ML. A. W. Appel. Cambridge University Press (1998).
- Introducción a la Programación Funcional con Haskell. Richard Bird. Prentice Hall (200)